

Week 4 Laboratory Assignment

Software Construction and Development

Part A Code Archaeology

The comments identify input, validation, calculation, grading and display in the original program.

```
def main():
    # INPUT
    name = input("Enter student name: ")
    marks = []
    for i in range(3):
        mark = float(input(f"Enter marks for subject {i + 1}: "))
        # VALIDATION and feedback DISPLAY
        while mark < 0 or mark > 100:
            print("Invalid marks. Enter 0-100.")
            # INPUT again after an invalid mark
            mark = float(input(f"Enter marks for subject {i + 1}: "))
        marks.append(mark)
    # CALCULATION
    total = sum(marks)
    average = total / len(marks)
    # GRADING / BUSINESS RULE
    if average >= 80:
        grade = "A"
    elif average >= 70:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 50:
        grade = "D"
    else:
        grade = "F"
    # DISPLAY
    print("\nStudent Result")
    print("-----")
    print("Name:", name)
    print("Total:", total)
    print("Average:", average)
    print("Grade:", grade)
main()
```

Part B Responsibility Map

Concern	Responsibility	Location
Input	Read the name and three numeric marks; repeat input when needed.	main.py
Validation	Decide whether a mark lies between 0 and 100 inclusive.	validation.py
Calculation	Calculate total and average from the marks.	calculations.py
Grading rule	Map the average to A, B, C, D or F.	calculations.py
Display	Print the result labels and values.	display.py

main.py prints the retry message. validation.py only returns True or False.

Part C Module Design

The project uses main.py, validation.py, calculations.py and display.py. verify.py contains the tests.

main.py collects input, retries invalid marks, calls the calculation functions and passes the result to display.py.

validation.py checks marks using the original range rule. Callers only need its Boolean result.

calculations.py contains calculate_total, calculate_average and calculate_grade. These functions return results without handling input or output. calculate_average reuses calculate_total.

display.py prints the supplied result. Grading belongs in calculations.py so presentation changes do not affect business rules.

Original Behaviour

The program accepts three numeric marks. Grades are A $\geq$ 80, B $\geq$ 70, C $\geq$ 60, D $\geq$ 50 and F below 50. The average is not rounded.

The original comparisons are preserved. Nonnumeric input still raises ValueError.

Part D Implementation

Save these files together in the student_marks folder. Python 3 and its standard library are sufficient.

validation.py

```
def validate_mark(mark):
    """Return True when a mark is within the allowed range."""
    return not (mark < 0 or mark > 100)
```

calculations.py

```
def calculate_total(marks):
    return sum(marks)

def calculate_average(marks):
    return calculate_total(marks) / len(marks)

def calculate_grade(average):
    if average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    return "F"
```

display.py

```
def display_result(name, total, average, grade):
    print("\nStudent Result")
    print("-----")
    print("Name:", name)
    print("Total:", total)
    print("Average:", average)
    print("Grade:", grade)
```

main.py

```
from validation import validate_mark
from calculations import (
    calculate_total, calculate_average, calculate_grade
)
from display import display_result
```

```
def main():
    name = input("Enter student name: ")
    marks = []
    for i in range(3):
        prompt = f"Enter marks for subject {i + 1}: "
        mark = float(input(prompt))
        while not validate_mark(mark):
            print("Invalid marks. Enter 0-100.")
            mark = float(input(prompt))
        marks.append(mark)

    total = calculate_total(marks)
    average = calculate_average(marks)
    grade = calculate_grade(average)
    display_result(name, total, average, grade)
```

```
if __name__ == "__main__":
    main()
```

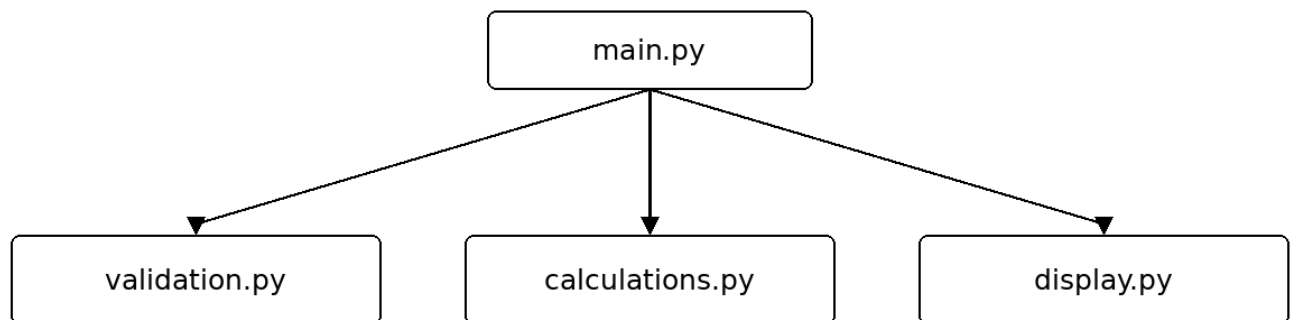
Running the Project

```
cd student_marks
python main.py
```

```
# Run the supplied checks
python verify.py
```

The main guard prevents input prompts when main.py is imported. Invalid attempts do not count as accepted marks.

Part E Dependency Mapping



Arrows show imports. main.py depends on the three supporting modules, which do not import one another or main.py.

display.py receives the finished grade. calculations.py returns values without knowing the output format.

A GUI would replace console presentation and require input wiring in main.py. Validation and calculation logic can stay unchanged.

The main.py imports are necessary. Imports between display.py and calculations.py would unnecessarily mix their responsibilities.

Part F Change Request Test

Change	Module affected	Reason
A ≥ 85, B ≥ 75, C ≥ 65, D ≥ 55	calculations.py	Only calculate_grade contains the thresholds.
Result format changes	display.py	display_result owns labels and layout.
Validation message changes	main.py	The retry loop prints the feedback message.
GUI output replaces console output	display.py; main.py for GUI wiring	Replace presentation and adapt interaction; reuse calculations and validation.

The code keeps the original thresholds. Under Change 1, an average of 82 changes from A to B; total and average calculations stay the same.

The retry message belongs to main.py; the allowed range belongs to validation.py.

Part G Verification

The table shows the verified results, including exact grade thresholds and invalid-input retries.

Test	Input	Verified result
Normal student	75, 82, 68	Total 225; average 75; grade B
Grade boundaries	50, 60, 70, 80	D, C, B, A respectively
Just below boundaries	49.99, 59.99, 69.99, 79.99	F, D, C, B respectively
Valid range limits	0 and 100	Both accepted
Invalid attempts	-5 and 105	Rejected; same subject is prompted again
Second student	40, 45, 35	Total 120; average 40; grade F

Actual Console Output

The following run shows rejected marks followed by valid replacements.

```
Enter student name: Ali
Enter marks for subject 1: -5
Invalid marks. Enter 0-100.
Enter marks for subject 1: 75
Enter marks for subject 2: 105
Invalid marks. Enter 0-100.
Enter marks for subject 2: 82
Enter marks for subject 3: 68
```

Student Result

```
-----
Name: Ali
Total: 225.0
Average: 75.0
Grade: B
```

Second student inputs: Sara; 40; 45; 35.

Enter student name: Sara
Enter marks for subject 1: 40
Enter marks for subject 2: 45
Enter marks for subject 3: 35

Student Result

Name: Sara
Total: 120.0
Average: 40.0
Grade: F

All five tests passed. The modular output also matched the original program in 13 comparison cases.

Reflection

Functions separate tasks; modules group related tasks in separate files with clear responsibilities.

validation.py has the strongest cohesion because its only task is checking marks.

main.py is coupled to calculate_grade through its name, argument and return value, but does not depend on its internal comparisons.

Modularity isolates changes: grading changes affect calculations.py, while result formatting changes affect display.py.

The four modules suit this program. validation.py is small but reusable. calculations.py would be too broad if it also handled input, storage or formatting.